

WEBER Loïc

DERRIDJ Mellyna



MICROPROCESSEUR – TP 2-3

BARE METAL

Question 1:

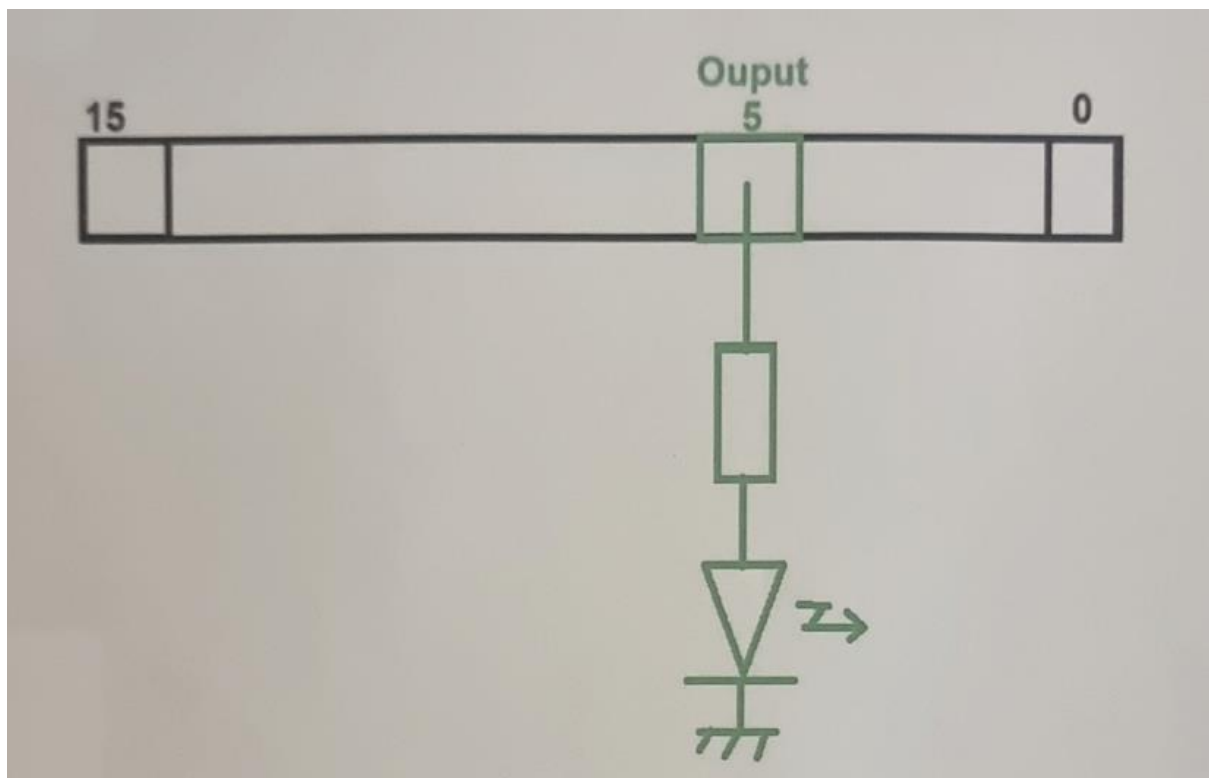


Schéma pour allumer une LED avec un 1 à partir d'une broche de sortie

On connecte l'anode de la LED à "output pin" du microcontrôleur, on connecte la cathode de la LED à une résistance (ce qu'on peut directement voir sur la carte Nucléo) qui sera branchée à la masse. La LED verte que nous utilisons est connectée aux pins PA5 et PB13 (la soudure étant faite sur le PA5 c'est celui-ci qui est utilisé).

doc explorée ?

Question 4 :

C'est le registre RCC (Reset and Clock Control) qui sert à activer des ports. ✓

La LED que nous utilisons dans ce TP se trouve sur le port GPIOA5 (Il aurait également pu se retrouver sur le port GPIOB13 qui peut aussi être relié à la LED, mais c'est le GPIOA5 qui a été connecté). Le bit relié à ce port est le bit 0.

de quel registre RCC ?

Question 5 :

```
LED_ENABLE:
    PUSH {R4,LR}
    LDR R0, =0x4802104C
    MOV R1, #1
    LDR R2, [R0]
    ORR R1,R1,R2
    STR R1, [R0]
    POP {R4,PC}
END_LED_ENABLE: B END_LED_ENABLE
```

Figure 1 : Code de la fonction LED_ENABLE

Question 6 :

Configurer le pin en push-pull permet de “contrôler” la LED en ayant une tension.

Pour configurer le pin il faut utiliser l'adresse du GPIO plus un offset en fonction du registre concerné :

MODER : +0x00 valeur en binaire : 01

OTYPER: +0x04 valeur en binaire : 0

OSPEEDR : +0x08 valeur en binaire : 00

PUPDR: +0x0C valeur en binaire : 00

ODR: +0x14

IDR: +0x10

On a donc en binaire : 01 0 00 00 (7 bits dans cet ordre logique). Pour les registres à 2 bits (MODER, OSPEEDR, PUPDR), les bits concernés sont 11 et 10. Pour OTyPER, ODR et IDR, c'est le bit 5.

Valeurs par défaut (Reset) : MODER : 0xABFF FFFF (Le port A est spécial car les pins de debug JTAG sont actives par défaut). Pour PA5, la valeur par défaut est 00 (Input).

OTYPER, PUPDR, OSPEEDR : Généralement 0 au reset pour PA5.

On modifie uniquement le registre MODER car les autres sont initialisés avec les bonnes valeurs.

Question 7 :

```
23
24 LED_Configure:
25     PUSH {R4,LR}
26     LDR R0, =0x48000000
27     LDR R1, [R0]
28     MOV R2, #0xFFFFF3FF
29     MOV R3, #0x00000400
30     AND R1, R2      //ON fait un ET entre R1 et R2
31     ORR R1,R1,R3    //On fait un OU entre R1 et R3
32     STR R1, [R0]
33     POP {R4,PC}
34 END_LED_Configure: B END_LED_Configure
35
```

Figure 2 : Code de la fonction LED_Configure

Comme dit dans la question précédente, les paramètres tels que OTYPE ou PUPD n'ont pas besoin d'être modifiés car ils sont initialisés avec la valeur que nous souhaitons.

Question 8 :

explication ?

```
35
36 LED_DriveGreen:
37     PUSH {R4, LR}
38 //   MOV R0, #1
39     AND R0, R0, #1
40     LDR R1, =0x48000014
41     MOV R2, #1
42     LSL R2, R2, #5
43     MOV R3, #0
44     LSL R3, R3, #5
45
46     CMP R0, #1
47     BEQ then_part
48
49 else_part:
50     STR R3, [R1]
51     B end_if
52
53 then_part:
54     STR R2, [R1]
55
56 end_if:
57     POP {R4, PC}
58
59 END_LED_DriveGreen: B END_LED_DriveGreen
60
```

Figure 3 : Code de la fonction LED_DriveGreen

Question 9 :

pas possible

```

39 AndAgain:
40
41 BL LED_DriveGreen //On appelle un première fois notre fonction
42 MOV R5, #1 //On initialise une valeur élevée dans un registre libre
43 LSL R5, R5, #18 //La valeur 18 peut être modifiée pour rallonger ou raccourcir la durée de la boucle
44 delay: //On crée une boucle de laquelle l'ordinateur mettra un certain temps à sortir
45 SUB R5, #1
46 CMP R5, #0
47 BNE delay
48 MOV R0, #1
49 BL LED_DriveGreen //On appelle notre fonction après avoir initialisé R0 à la bonne valeur
50 MOV R5, #1
51 LSL R5, R5, #18 //La valeur 18 peut être modifiée pour rallonger ou raccourcir la durée de la boucle
52 delay2: //On refait le même type de boucle que précédemment
53 SUB R5, #1
54 CMP R5, #0
55 BNE delay2
56 MOV R0, #0
57
58 B AndAgain
59

```

je

Figure 4 : Code ayant permis de faire cligotter la LED

Cette question n'a pas été réalisée selon ce que le sujet demandait (de lier le code ASM à un code en C) car cela nous a tout d'abord semblé contre intuitif et nous ne comprenions pas que cela était possible et comment cela pouvait fonctionner. Par la suite nous avons demandé à l'enseignant encadrant comment faire et il nous l'a expliqué et aidé à codé, cependant la variable globale demandait ne semblait pas fonctionner comme voulu sans que lui ou nous ne comprenne pourquoi.

Etant donné que la séance approchait de la fin, nous avons choisi de privilégier de poursuivre le TP avec la méthode que nous avons codé précédemment pour faire clignoter la LED afin de pouvoir apprendre plus sur le sujet plutôt que de le finir en essayant de résoudre un problème que le professeur ne comprenait pas non plus.

Cependant notre méthode ne nécessitant pas de variable globale et étant éloignée de ce qui était demandé à la base, les questions 10, 11 et 12 n'ont pas pu être traitées sur machine. Nous avons donc lu et compris ce qui était attendu de nous sans pour autant pouvoir vérifier sur machine la manière dont nous souhaitions aborder ces questions.

Le code que nous avons écrit et utilisé étant éloigné de ce qui était demandé, nous allons brièvement l'expliquer ici :

Nous avons écrit dans une boucle infinie un code alternant un appel de LED_DriveGreen allumant et éteignant la LED. Cependant l'ordinateur faisant ces opérations trop rapidement, il nous a fallu utiliser l'équivalent d'un DELAY en C. Cet équivalent n'existant pas directement (en tout cas

pas à notre connaissance (malgré des recherches sur internet) ou celle du professeur encadrant) nous avons codé une succession d'un très grand nombre d'opération entre les appels de LED_DriveGreen pour empêcher l'ordinateur d'enchaîner trop rapidement les appels de la fonction. Ces successions d'opérations consistaient simplement à décrémenter une valeur en boucle jusqu'à atteindre 0. En ajustant la taille de la valeur nous pouvions donc contrôler avec relativement de précision la durée d'attente entre chaque appel de fonction et donc la fréquence à laquelle la LED clignotait.

Question 14 :

Le bouton se trouve sur le pin GPIOC13.